Looker E-commerce: End-to-End Data Engineering Pipeline

Final report: design decisions, challenges and results

1. Executive summary

This project builds an automated data platform for an e-commerce company that suffered from slow reporting, unclean data, and no real-time view of customer behavior. It uses the Looker E-commerce BigQuery dataset and a **Medallion architecture** (bronze, silver, gold).

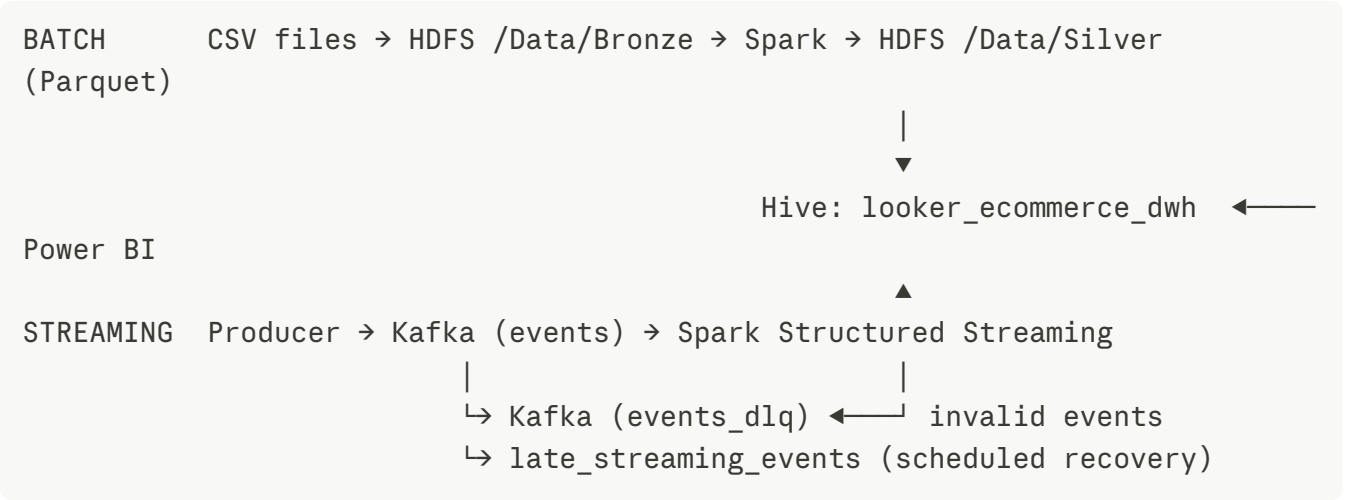
- **Batch path:** Local files → HDFS (bronze) → Spark cleaning (silver) → Hive star schema (gold).
- **Streaming path:** a simulated clickstream → Kafka → Spark Structured Streaming → the same Hive `fact_events` table.
- **Orchestration:** an Airflow DAG runs the batch layers only when the source files change, and recovers late events on every run.
- **Consumption:** a six-page Power BI report on top of the warehouse.

The design goals were correctness first (no duplicates, no lost events, no overwritten history), then automation, then performance. Every load step is idempotent, so the pipeline can be re-run safely.

2. Problem statement

Area	Problem	How the project addresses it
Batch	Slow reporting	Dimensional model in Hive with partitioning, bucketing and Parquet/Snappy
Batch	Unclean data	Profiling in bronze, rule-based cleaning in silver, quarantine of invalid rows
Streaming	Need for real-time analytics	Kafka plus Spark Structured Streaming, micro-batches every 5 minutes into Hive
Streaming	Duplicate and late-arriving events	Watermark, key-based de-duplication, dead-letter topic, scheduled late-event recovery

3. Architecture



Technology stack: HDFS, Apache Spark (PySpark, Structured Streaming), Hive, Apache Kafka (3 brokers), Apache Airflow, Jupyter notebooks (executed headlessly with `nbconvert`), Docker, Power BI.

4. Phase 1: Data ingestion (bronze)

What was done. The `bronze_layer` notebook uploads the five raw CSVs (`users`, `products`, `distribution_centers`, `order_items`, `events`) from the local file system to HDFS using Python, and then verifies that every expected file is present in the bronze folder. A missing file raises an error and stops the pipeline.

Decision: bronze stays raw. No data is changed in bronze. The files are copied exactly as received, so any later problem can be traced back to the source.

Data-quality analysis. After loading, each dataset is profiled with reusable checks:

- null counts per column
- duplicate keys
- distinct values of categorical columns (gender, status, browser, traffic source, event type)
- out-of-range values (age, prices)
- timestamps in the future
- date order across the order lifecycle (created → shipped → delivered → returned)
- orphan keys (products without a distribution center, orders without a user or product, events without a user)

The checks are informational: they print findings for the engineer. The decision on what is invalid is enforced in silver.

5. Phase 2: Data cleaning (silver)

Schemas. Each file is read with an explicit schema instead of `inferSchema`. This avoids a second pass over the data and prevents silent type drift (for example, `user_id` in events is read as a double because of nulls, then cast to an integer).

Standardization. Per dataset, the notebook:

- trims and collapses whitespace
- strips unexpected characters from names and places
- applies consistent casing (`initcap`, `upper`, `lower`)
- lower-cases emails
- casts money to `DECIMAL(10,2)`
- derives the distribution-center state from its name
- maps order status names to numeric IDs (Processing 0, Shipped 1, Complete 2, Returned 3, Cancelled 4, Unknown -1)
- drops columns with no analytical value (latitude, longitude, postal code, SKU, IP address, inventory item ID)

Handling nulls. Missing values are handled by column role rather than with one blanket rule. Descriptive attributes get a clear placeholder (`Unknown` for city, product name and brand). Missing keys, timestamps and prices are treated as invalid data, not filled in, because inventing a value would corrupt the facts.

Quarantine of invalid records. A single function, `split_invalid`, applies a validity rule per dataset. Rows that fail the rule (or where the rule evaluates to null) are written to `/Data/Silver/Invalid_…` as Parquet instead of being deleted, as the brief requires ("remove invalid records for future investigation"). Examples of rules: age between 0 and 100, gender M or F, positive prices below 100,000, and an ordered lifecycle for order dates.

The `reading_invalid_data` notebook re-reads the quarantine and attaches a human-readable reason to every row.

Key finding. 35,872 order items were quarantined, **all for the same reason: `shipped_at` earlier than `created_at`**. This is a property of the source data, not a pipeline bug. It is a large share of the order items, so revenue and order counts in the dashboard cover only the validated records. The decision to quarantine rather than "repair" these dates was deliberate, because there is no way to know which of the two timestamps is wrong.

Output. Clean data is written to `/Data/Silver/Cleaned_…` as Parquet, one folder per entity, with a controlled number of files.

6. Phase 3: Dimensional model and Hive load (gold)

6.1 Schema

A star schema in the database `looker_ecommerce_dwh`:

Table	Type	Grain / purpose	Physical design
`fact_order_items`	Fact	One row per order item, with four date keys (created, shipped, delivered, returned) and a status	Partitioned by `order_year`, 4 buckets on `user_key`
`fact_events`	Fact	One row per web event	Partitioned by `event_year`, `event_month`
`dim_users`	SCD2 dimension	One row per user version	Partitioned by `traffic_source`, 4 buckets on `user_key`
`dim_products`	SCD2 dimension	One row per product version, with the distribution center denormalized in	4 buckets on `product_key`
`dim_order_status`	Dimension	Status ID to name	Single small table
`dim_date`	Dimension	One row per calendar day, from the earliest data year to the year after the latest	Single small table

All tables are stored as **Parquet with Snappy compression**. Primary and foreign keys are declared as informational constraints (`DISABLE NOVALIDATE RELY`), which Hive uses for query planning without the cost of enforcement.

6.2 Design decisions

- **Surrogate keys.** Hive has no auto-increment, so new keys are `max(existing) + row_number()`.
- **Unknown members.** `user_key = 0`, `product_key = 0` and `status_id = -1` receive anonymous events and unmatched records. Facts therefore never have a null or broken foreign key, and no fact row is lost because of a failed join.

- **Role-playing date dimension.** `fact_order_items` links to `dim_date` four times, once per lifecycle event, which supports delivery-time and return analysis directly.
- **Slowly Changing Dimension Type 2 for users and products.** Tracked attributes (users: names, age, state, address, city, country; products: name, cost, price, distribution center) create a new version when they change, so history is preserved. The first version of a user starts on the registration date, and the first version of a product starts on its first order date. This lets historical facts join to the version that was valid on their date.
- **Point-in-time joins.** `attach_scd2_key` assigns each fact the surrogate key of the dimension version valid on the fact's date, with fallbacks to the current version and then to the unknown member. It reads the dimension fresh on each call, so it also works inside streaming micro-batches.
- **Partitioning and bucketing.** Partitions follow how the data is queried and grows (by year for orders, by year and month for events). Bucketing on the user key reduces shuffle for the common user-level joins. File counts are controlled at write time (one file per partition per run), which avoids the small-files problem.

6.3 Reusable loaders (`utils`)

Loader	Behavior	Used for
`scd2_load`	Detects new and changed records, expires old versions, inserts new ones; does nothing when nothing changed	`dim_users` , `dim_products`
`upsert_fact`	Replaces only the partitions touched by the batch, merging new rows with untouched old rows	`fact_order_items`
`simple_load`	Inserts only keys that are not yet in the table; never overwrites	`fact_events` , `dim_date` , `dim_order_status`

Hive tables cannot be updated in place, so the SCD2 and upsert loaders rebuild the affected data in a temporary staging table, overwrite the target from it, and always drop the staging table, even on failure.

7. Phase 4: Kafka simulation

`kafka_producer.py` simulates live user sessions to a 3-broker Kafka cluster (topic `events`). The producer is configured for reliability: `acks=all` , retries, and one in-flight request per connection, with its own connection retry logic.

Realism. Each session follows a funnel (Home → Department → Product → Cart → Purchase), and 20% of completed funnels end in a cancel instead. Events are stamped with the real send time, so the stream behaves like production traffic and the watermark advances normally. Anonymous users are included (about one in six sessions).

Deliberate problems. To prove the pipeline handles the required challenges, the producer injects them at configurable rates:

Problem	Default rate	How it is injected
Duplicates	5%	An already-sent event is sent again with the same `event_id`
Late events	5%	`created_at` is 1 to 5 minutes in the past (inside the 1-hour watermark)
Very late events	2%	`created_at` is 61 to 180 minutes in the past (outside the watermark)
Invalid events	2%	Missing ID, sequence number ⩽ 0, future date, unparseable date, or not JSON at all

Decision: synthetic events. The producer generates events instead of replaying `events.csv`, because replaying historical rows with their original timestamps would be "very late" by definition and would not exercise the real-time path.

8. Phase 5: Spark Structured Streaming

8.1 Main stream

1. **Read** from Kafka (`events`) with `failOnDataLoss=false`.
2. **Parse** the JSON with an explicit schema. Malformed JSON becomes a row of nulls instead of crashing the stream.
3. **Clean** with the same rules as silver, so batch and streaming data are consistent.
4. **Validate** with one shared rule (`valid_event`): event ID present, parseable timestamp not in the future (1-minute tolerance), and sequence number ⩾ 1.
5. **Late-data policy.** Events more than 1 hour older than their arrival time are excluded, and a 1-hour watermark bounds the state kept for de-duplication.
6. **De-duplicate** on `event_id` and `created_at` within the watermark.
7. **Enrich** with the point-in-time user key (anonymous users map to the unknown member).

8. **Load** with `foreachBatch` every 5 minutes into `fact_events` through `simple_load`, an insert-only loader that checks only the touched partitions for existing keys.

8.2 Dead-letter queue

A second stream reads the same topic, keeps only events that fail `valid_event`, and writes their raw payload to the Kafka topic **`events_dlq`**. Nothing invalid reaches the warehouse, and nothing invalid is lost either. The `reading_invalid_events_dlq` notebook explains each rejected message.

8.3 Recovery of very late events

Events older than the watermark are dropped by the stream on purpose, because unbounded lateness would mean unbounded state. They are recovered by `late_streaming_events`, a scheduled batch job that:

- re-reads the last 3 days of the topic
- applies the same clean and validate rules
- keeps events older than 2 hours (so it never competes with the live stream)
- de-duplicates and inserts only what is missing

Because the stream and the recovery job share the same insert-only loader, they can write to `fact_events` at the same time without overwriting each other.

8.4 Measured results

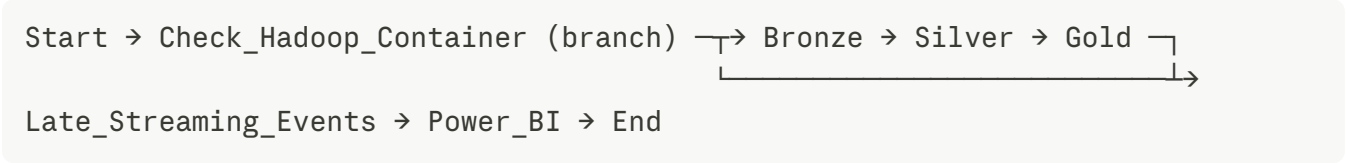
From the `reading_late_streaming_events` and DLQ notebooks (one test run):

Measure	Result
Messages in the `events` topic	6,222 (5,962 unique payloads)
Valid events on time	5,354 (92.6%)
Late events accepted by the stream	260 (4.5%), average delay 3.1 min; all present in `fact_events`
Very late events dropped by the stream	170 (2.9%), delay 63 to 180 min
Very late events already recovered by the late job	15 of 170
Very late events waiting for the next recovery run	155 (105 already eligible, 50 not yet 2 hours old)
Unique invalid events in the DLQ	178

DLQ breakdown: sequence number ⩽ 0 (39), future `created_at` (38), unparseable date (36), not valid JSON (33), missing `event_id` (32). These match the five fault types the producer injects, which confirms the validation rules work.

9. Orchestration with Airflow

The DAG `looker_ecommerce_pipeline` runs daily. Each notebook runs inside the Hadoop container via the Docker API, and a failing command fails the task. The executed notebook is saved per day as an audit trail and data-quality report.



- **Change detection.** The branch task fingerprints the source files (name, size, modified time) and compares them with the last successful run, which is stored in an Airflow Variable. If nothing changed, bronze, silver and gold are skipped. Late-event recovery still runs.
- **Safe fingerprint.** The fingerprint is saved only after the gold layer succeeds. A failed run is therefore retried in full on the next trigger.
- **Reliability.** One retry with a 5-minute delay, a 45-minute task timeout, a 4-hour DAG timeout, a single active run at a time, and e-mail on failure.
- **Trigger rule.** `none_failed_min_one_success` lets the late-event task run whichever branch was taken.

The two DAG-run screenshots show both paths: a first run where every step succeeds, and a second run where the three batch layers are skipped and only late-event recovery runs.

10. Challenges and how they were solved

Challenge	Solution
Duplicate events	Producer-injected duplicates are removed in the stream (watermarked de-duplication on `event_id` and `created_at`). The insert-only loader skips keys that already exist, and the recovery job de-duplicates by `event_id`. Three layers protect the table.
Late-arriving data	Three tiers: on-time and slightly late events go straight through; events inside the 1-hour watermark are accepted; older events are dropped by the stream and recovered by the scheduled job.
Avoid overwriting data	The streaming and recovery paths are append-only. Dimension and fact upserts overwrite only what they must (the SCD2 table, or the partitions touched by the batch), and they merge existing rows back in, so no history is lost.
Data consistency	Shared cleaning and validation rules across silver, stream and recovery; point-in-time dimension joins; unknown members so no key is null; informational PK/FK constraints; invalid data is quarantined, not silently dropped.
Updates in a store without UPDATE	Rebuild through a staging table, then overwrite; the staging table is always cleaned up. Static or dynamic partition overwrite is set explicitly on every write so one loader cannot inherit another's mode.
Surrogate keys without auto-increment	`max(key) + row_number()`, with key 0 reserved for the unknown member.

Challenge	Solution
Source data quality	35,872 order items with shipping before creation were quarantined and documented. They were not repaired, because the correct value is unknowable.

11. Bonus: idempotency

Running the pipeline repeatedly does not duplicate data:

Component	Why a re-run is safe
Bronze	`hdfs dfs -put -f` overwrites the same files with the same content
Silver	Each output folder is overwritten with the same result; the quarantine path is cleared and rewritten
`dim_users`, `dim_products`	Only new or changed business keys produce rows; an unchanged load prints "nothing to load"
`fact_order_items`	Upsert by `order_item_id` replaces only touched partitions, so a re-run yields the same rows
`dim_date`, `dim_order_status`, `fact_events`	Insert-only for keys that do not exist yet
DAG	Unchanged sources skip the batch layers entirely

Two screenshots demonstrate this behavior on real data:

- **SCD2:** user 457's city changed from Rio Branco to Sena Madureira. The old row was closed (`end_date` set, `is_current` false) and a new row was created with `user_key` 100,001.
- **Fact upsert:** order item 36,717 changed from Shipped (status 1) to Complete (status 2) and gained a delivered date key. The row was updated in place rather than duplicated.

12. Reporting layer (Power BI)

A six-page Power BI report reads from the warehouse: Executive, Products, Customers, Geography, Operations and Web.

13. Conclusion

The project delivers every phase in the brief: ingestion, cleaning, dimensional modeling with partitioning and bucketing, a Kafka simulation, and a streaming path that handles duplicates and late data. It adds an Airflow DAG, a dead-letter queue, a late-event recovery job and a Power BI report. The central design principle is that data is never silently lost or overwritten: invalid records are quarantined or dead-lettered, history is preserved with SCD2, and every load can be repeated safely.

Appendix: deliverables

File	Purpose
`bronze_layer.ipynb`	Upload to HDFS and data profiling
`silver_layer.ipynb`	Cleaning, schemas and quarantine
`gold_layer.ipynb`	Hive star schema load
`utils.ipynb`	SCD2, upsert and insert-only loaders; point-in-time join
`streaming_layer.ipynb`	Kafka to Hive stream and dead-letter stream
`late_streaming_events.ipynb`	Recovery of events older than the watermark
`reading_*.ipynb`	Inspection of quarantine, DLQ and late events
`kafka_producer.py`	Clickstream simulator
`looker_ecommerce_dwh.sql`	Hive DDL
`looker_ecommerce_pipeline_dag.py`	Airflow DAG
`Looker_Ecommerce_Power_BI.pdf`	Dashboard export